

Lekce 18: Cypress Dashboard, Reportování a Nepřetržitá Integrace a Dodávání (CI/CD)

Cíle

- **Cypress Dashboard:**
Porozumění funkcím a výhodám Cypress Dashboard pro správu a sledování spouštění testů.
- **Reportování:**
Naučit se vytvářet, přizpůsobovat a interpretovat Cypress testovací reporty a metriky.
- **Integrace CI/CD:**
Integrace Cypress testů do automatizovaných pracovních postupů pomocí populárních CI/CD nástrojů (například GitHub Actions, GitLab CI nebo Jenkins), aby se testy spouštěly při každém commitu a nasazení kódu.

Rozvržení obsahu

A. Cypress Dashboard

1. Přehled funkcí Cypress Dashboard:

Webová služba poskytovaná [Cypress.io](https://dashboard.cypress.io) pro správu a vizualizaci testovacích běhů. Dostupná pro open-source i komerční použití.

Registrace na <https://dashboard.cypress.io>

- **Monitorování v reálném čase:**
Zobrazte stav testovacích běhů, videa, snímky obrazovky a logy v reálném čase.
- **Historie testů & analýzy:**
Analyzujte trendy v čase, prohlédněte si detailní historii běhů a sledujte výkon.
- **Ladění chyb:**
Přístup k "time-travel" snímkům a detailním logům pro rychlou identifikaci selhání.
- **Spolupráce:**
Sdílejte výsledky testů s členy týmu a integrujte s nástroji pro sledování chyb.

2. Výhody pro správu testů a reportování:

- **Centralizovaný pohled:**
Všechny běhy a jejich detaily jsou uloženy na jednom dashboardu.

- **Automatické pořizování videí/snímků:**

Jednodušší ladění nekonzistentních problémů.

- **Rozšířené reportování:**

Vizualizace výkonu testů a trendů.

- **Týmová spolupráce:**

Usnadňuje sdílení poznatků a historických dat s účastníky projektu.

3. Nastavení integrace Cypress Dashboard:

- **Vytvoření účtu:**

Registrujte se na [Cypress Dashboard](#).

- **Konfigurace projektu:**

Přidejte ID projektu do konfigurace Cypress.

```
// cypress.config.js ukázka:
module.exports = defineConfig({
  e2e: {
    // ... další nastavení
    projectId: 'your-project-id', // Dodáno Cypress Dashboard
  }
});
```

- **Nahrávání výsledků:**

Použijte CLI příkaz `npx cypress run --record --key <record-key>` pro nahrání výsledků testů.

- **Přidání do testovacího příkazu:**

```
"scripts": {
  "test:cloud": "cypress run --record --key YOUR_PROJECT_KEY"
}
```

B. Reportování

1. Generování testovacích reportů:

- Pomáhá detekovat vzorce selhání.
- Dokumentuje kvalitu a spolehlivost testů.
- Užitečné pro týmovou komunikaci a CI dashboardy.
- Snímky obrazovky a videa jsou automaticky ukládány při selhání (`videos/` , `screenshots/`).
 - **Vestavěné reportování:**
Cypress poskytuje základní reportování v CLI, včetně podrobností o úspěšných,

neúspěšných a přeskočených testech.

- o **Vlastní reportery:**

Pluginy jako `cypress-mochawesome-reporter` mohou generovat detailnější HTML reporty.

```
npm install cypress-mochawesome-reporter --save-dev
```

Poté nastavte ve své Cypress konfiguraci:

```
module.exports = defineConfig({
  reporter: 'cypress-mochawesome-reporter',
  reporterOptions: {
    charts: true,
    embeddedScreenshots: true,
    inlineAssets: true,
  },
  e2e: {
    setupNodeEvents(on, config) {
      require('cypress-mochawesome-reporter/plugin')(on);
    }
  }
});
```

Přidejte do `cypress/support/e2e.js`

```
import 'cypress-mochawesome-reporter/register'
```

2. Přizpůsobení výstupů reportů:

- Přizpůsobte název reportu, téma a cesty pro výstup pomocí volby reporter options.
- Analyzujte metriky jako délka testu, počet opakování a logy chyb pro získání statistik.

3. Analýza výsledků testů a metrik:

- Prohlížejte snímky obrazovky a video záznamy poskytnuté Dashboardem.
- Využijte detailní logy a snímky pro přesné určení selhání a výkonových problémů.

C. Nepřetržitá integrace (CI) a nepřetržité dodávání (CD)

- **CI (Continuous Integration):** Automatické testování a ověřování změn v kódu.
- **CD (Continuous Delivery/Deployment):** Automatické nasazení kódu do testovacího nebo produkčního prostředí.

♦ Oblíbené CI nástroje:

- GitHub Actions 
- GitLab CI
- Jenkins
- CircleCI
- Bitbucket Pipelines
- Azure DevOps

1. Úvod do pojmů CI/CD:

- **CI (Continuous Integration):**

Automatizace procesu sestavení a testování kódu při každém commitu.

- **CD (Continuous Delivery):**

Automatizace nasazení tak, aby byly změny co nejrychleji a spolehlivě doručeny do produkce.

2. Nastavení běhu Cypress testů v CI/CD pipeline:

- **Příklad GitHub Actions:**

name: Cypress E2E Tests

on:

push:

branches: [main, master]

pull_request:

branches: [main, master]

workflow_dispatch:

inputs:

spec:

description: 'Spec soubor(y) ke spuštění (volitelné, spustí všechny pokud prázdné)'

required: false

default: ''

type: string

browser:

description: 'Prohlížeč pro spuštění testů'

required: false

default: 'chrome'

type: string

jobs:

cypress-run:

name: Spustit Cypress testy na \${{ matrix.browser }}

runs-on: ubuntu-latest

strategy:

fail-fast: false

matrix:

browser: [chrome, firefox, edge]

steps:

- name: Checkout kódu

uses: actions/checkout@v4

- name: Nastavení Node.js

uses: actions/setup-node@v4

with:

node-version: '18'

cache: npm

- name: Instalace závislostí

run: npm ci

- name: Ověření Cypress binary

run: npx cypress verify

```

- name: Spuštění Cypress testů
  uses: cypress-io/github-action@v6
  with:
    browser: ${{ matrix.browser }}
    spec: ${{ github.event.inputs.spec }}
    record: true
    group: 'E2E na ${{ matrix.browser }}'
    tag: ${{ github.event_name }}
    parallel: false
  env:
    CYPRESS_RECORD_KEY: ${{ secrets.CYPRESS_RECORD_KEY }}
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

- name: Nahrát snímky při selhání
  if: failure()
  uses: actions/upload-artifact@v4
  with:
    name: screenshots-${{ matrix.browser }}
    path: cypress/screenshots

- name: Nahrát videa při selhání
  if: failure()
  uses: actions/upload-artifact@v4
  with:
    name: videos-${{ matrix.browser }}
    path: cypress/videos

```

- **Automatizace běhu testů:**

Nastavte vaše CI/CD pipeline tak, aby se testy spouštěly automaticky při commitech, pull requestech nebo nasazeních.

3. Notifikace a upozornění:

- Integrate se Slackem, emailem či dalšími komunikačními systémy pro upozornění týmu na selhání testů.
- Využijte upozornění Dashboardu pro informování stakeholderů o problémech buildu.

4. Osvědčené postupy v CI/CD:

- Bezpečně spravujte proměnné prostředí a tajné klíče (použijte šifrované úložiště v CI/CD platformě).
- Zajistěte konzistenci testovacího prostředí například pomocí Dockeru nebo kontejnerizovaných řešení.

- Optimalizujte spouštění testů paralelizací a použitím cache strategií, kde je to možné.

D. Praktické aktivity

1. Nastavení Cypress Dashboard:

- Vytvořte si účet na Cypress Dashboard a nakonfigurujte projekt pomocí přiděleného project ID.
- Spusťte testy s volbou record pro nahrání výsledků do Dashboardu.

2. Konfigurace CI pipeline:

- Použijte GitHub Actions (nebo jiný CI nástroj) pro nastavení workflow, které automaticky spustí Cypress testy při push a pull requestech.
- Procvičte si spouštění pipeline a kontrolu vygenerovaných reportů.

3. Vytváření a interpretace testovacích reportů:

- Nakonfigurujte vlastní reporter (např. `cypress-mochawesome-reporter`) a generujte HTML reporty.
- Analyzujte reporty pro identifikaci nestabilních testů, výkonových problémů a trendů chyb.

4. Simulace scénáře z praxe:

- Použijte `cy.intercept()` k podvržení síťových požadavků a simulujte různé reakce backendu.
- Kombinujte UI testy s API testy tak, že ověříte konzistenci dat zobrazovaných v UI s odpověďmi API.

5. CI/CD Notifikace:

- Nakonfigurujte notifikace (například přes Slack) v CI pipeline pro upozornění na neúspěšné testy.
- Diskutujte, jak automatizovaná oznámení zlepšují reakci týmu na vzniklé problémy.

E. Zdroje

• Dokumentace Cypress Dashboard:

[Cypress Dashboard](#)

• Reportovací pluginy pro Cypress:

- [cypress-mochawesome-reporter](#)

• CI/CD integrační návody:

- GitHub Actions: [GitHub Actions for Cypress](#)
- GitLab CI: [Cypress with GitLab CI](#)

- Jenkins: [Using Cypress with Jenkins](#)

- **Komunitní příklady:**

Vyhledejte na GitHubu repozitáře používající Cypress s CI/CD pipeline pro další příklady a inspiraci.

Možné studentské otázky a odpovědi

1. **Q:** *Jaký je hlavní přínos využití Cypress Dashboard?*

A: Cypress Dashboard centralizuje výsledky testovacích běhů, nabízí zpětnou vazbu v reálném čase pomocí videí a snímků obrazovky a pomáhá při ladění chyb díky detailním logům a historickým datům.

2. **Q:** *Jak mohu přizpůsobit reporty testů v Cypress?*

A: Můžete použít vlastní reportery jako `cypress-mochawesome-reporter` pro generování detailních HTML reportů. Tito reportéři se nastavují prostřednictvím volby options v konfiguračním souboru Cypress a lze s nimi upravit formát i umístění výsledných reportů.

3. **Q:** *Co je CI/CD a proč je důležité pro automatizaci testů?*

A: CI/CD znamená Nepřetržitou Integraci a Dodávání. Automatizuje proces sestavení, testování a nasazení, což zajišťuje, že změny v kódu jsou automaticky ověřovány a rychle nasazeny. Integrace Cypress testů do CI/CD pipeline pomáhá odhalit chyby včas a zajistit kvalitu každého vydání.

4. **Q:** *Jak fungují proměnné prostředí a tajné klíče v CI/CD pipeline?*

A: CI/CD platformy poskytují bezpečný způsob správy proměnných prostředí a tajných údajů. Tyto hodnoty jsou vkládány do procesu sestavení a testování, aby se citlivá data, např. API klíče, nevyskytovala přímo v testech.

5. **Q:** *Jak může `cy.intercept()` zlepšit spolehlivost testů?*

A: `cy.intercept()` umožňuje podvrhnout síťové požadavky, což znamená, že vaše testy nejsou ovlivněny vnější variabilitou backendu. To vede k předvídatelnějším výsledkům testů a rychlejšímu provedení.

6. **Q:** *Jaké jsou výhody integrace Cypress testů do CI/CD pipeline?*

A: Automatizace spouštění testů při každém commitu či nasazení, rychlá zpětná vazba ke změnám v kódu, lepší spolupráce týmu a schopnost včas zachytit chyby jsou hlavními výhodami. Také umožňuje kontinuální zajišťování kvality v celém vývojovém cyklu.